Machine Saathi: Graph-Based Machine Failure Impact and Dependency Analyzer

1. Problem Statement

Every industrial machine, whether a compressor, lathe, conveyor, pump set, or CNC unit, consists of components that depend on one another. For example, a bearing supports a rotating shaft, and the shaft transfers mechanical motion within the machine. When a component fails, other components that depend on it may also be affected, either directly or indirectly.

In many workshops, these dependencies are documented in technical manuals or rely on the experience of maintenance technicians. When a failure occurs, particularly in older machines with incomplete documentation, identifying the components that may be affected and tracing their dependency paths can be challenging.

This project develops a machine-independent tool that represents machine components as vertices in a directed graph and their dependencies as edges. Using Breadth-First Search (BFS) and Depth-First Search (DFS), the system identifies potentially affected downstream components, determines their graph-distance levels, and traces possible dependency paths from a reported faulty component. This provides a structured and interpretable approach to machine failure impact analysis.

2. Motivation

Many factories still depend on legacy machines, and replacing them with modern, sensor-equipped equipment can be costly and disruptive. The wider Machine Saathi idea aims to make such machines easier to maintain without replacing them. This mini-project implements one foundational part of that idea: a structured and interpretable model of the dependencies between a machine's components.

A graph is suitable for this problem because machine structures are not always simple chains. One component may affect several others, while a component may depend on multiple components. Unlike a linear structure, a directed graph can represent these interconnected relationships. Breadth-First Search (BFS) identifies downstream components level by level, while Depth-First Search (DFS) traces possible dependency paths. These techniques provide explainable results without requiring machine learning, sensors, or real-time data.

Graph-based dependency and fault-propagation modelling is also an established research area in industrial maintenance [1], [3], providing academic context for this approach. This project is a simplified, data-structures-level implementation of dependency and failure-impact analysis, not a claim of developing new fault-diagnosis technology.

3. Objectives

1. Represent machine components as vertices and their dependencies as directed edges.
2. Use an adjacency-list representation to store component dependencies.

3. Allow users to add components and dependencies at runtime, enabling the tool to model different types of machines within the implementation’s capacity.
4. Identify downstream components level by level and distinguish direct impact from indirect impact using graph distance.
5. Explore and display possible paths from a faulty component to downstream components.
6. Store component health status as Operational or Faulty and display downstream components identified by BFS as At Risk.
7. Offer a menu-driven system for configuring machines, recording faults, analysing dependencies, searching components, and resetting component statuses.

4. Methodology

Graph Model: The system represents machine components as vertices in a directed graph, with directed edges indicating dependency relationships. The graph is implemented using an adjacency-list representation, which stores connections efficiently for sparse graphs. This structure supports graph traversal and dependency analysis using BFS and DFS.

Machine Independence: The program allows users to create machine configurations by adding components and defining their dependencies at runtime. It supports a maximum of 50 components and includes a preloaded configuration for demonstration purposes.

Failure Reporting and Impact Analysis: When a component failure is reported, the program updates the component's stored status to Faulty and enables downstream impact analysis. BFS identifies reachable downstream components and displays them as At Risk, while DFS traces possible dependency paths. The analysis relies on user-defined dependency relationships rather than sensor data or machine-learning predictions.

BFS for Impact Levels: BFS uses a queue and a visited array to explore reachable components level by level. It assigns graph-distance levels based on the shortest path from the selected starting component, helping distinguish direct dependencies from indirect dependencies. These levels indicate graph distance rather than actual failure timing.

DFS for Dependency-Path Tracing: DFS recursively explores downstream dependencies and uses backtracking to examine alternative branches. The program displays maximal simple paths discovered from the selected starting component. These paths represent possible dependency routes and do not establish the actual physical sequence of failure propagation. Path enumeration may require exponential time in the worst case.

Robustness: The program uses visited-state tracking during graph traversal to prevent repeated processing and to handle cycles appropriately. It validates component inputs and rejects invalid, duplicate, or self-referential dependencies according to the implemented checks.

Dependency Lookup: The program identifies components that directly depend on a selected component by scanning the graph's adjacency lists for incoming relationships. This operation provides direct dependency information but does not recursively identify all indirect upstream components or determine a definitive root cause.

Menu-Driven Interface: The program provides a menu-driven interface for managing machine components and dependencies, displaying the graph and machine health, searching components,

reporting faults, performing BFS impact analysis and DFS path tracing, looking up dependencies, changing machine configurations, resetting component statuses, and exiting the application.

5. Hardware and Software Used

Hardware: A standard computer or laptop capable of compiling and running a C program. No sensors or external hardware devices are required because the project focuses on graph-based dependency and failure-impact analysis.

Software: C programming language, GCC compiler, and Visual Studio Code on Windows.

6. Operations Implemented

Graph Creation and Updates: The program initializes the graph, adds components as vertices, and represents dependencies using adjacency-list edges. Component and dependency operations involve component-name lookups and validation, so their overall execution time is not strictly constant. Adjacency-list insertion itself is constant-time when the relevant vertex is already known and the new edge is inserted at the head of its list.

Component Search: The program searches for a component by name and displays its name, criticality value, and stored status. It does not display the number of dependent components as part of this search operation.

BFS Impact Analysis: Starting from the selected faulty component, BFS marks the starting vertex as visited, assigns it level 0, and inserts it into the queue. It repeatedly removes a vertex from the queue and visits its unvisited downstream neighbours, assigning each a level one greater than the current vertex. The process continues until the queue is empty. The output identifies reachable downstream components, their graph-distance levels, and impact-related summary information. Its traversal complexity is (O(V+E)), with (O(V)) auxiliary space for the visited information, levels, and queue.

DFS Dependency-Path Tracing: DFS starts from the selected component, marks vertices during the current path, and explores downstream neighbours recursively. Backtracking allows alternative simple paths to be explored. The program displays the paths discovered by this procedure. Standard DFS traversal has (O(V+E)) complexity, but enumerating all simple paths can take exponential time in the worst case. The recursion stack and current path require up to (O(V)) auxiliary space, excluding the output produced by path enumeration.

Status Management: The program stores two component statuses: Operational and Faulty. During BFS analysis, reachable downstream components are displayed as At Risk, but this is an analysis label rather than a third stored status. The reset operation restores component statuses to Operational.

Complexity Summary: The adjacency-list graph representation requires (O(V+E)) space. BFS traversal takes (O(V+E)) time and (O(V)) auxiliary space. A standard DFS traversal takes (O(V+E)) time, while the implemented path-enumeration procedure may require exponential time in the worst case. Component-name searches and dependency insertion operations also incur lookup costs.

7. Executable Code

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>
```

```
#include <stdbool.h>
#include <ctype.h>

#define MAX_VERTICES 50
#define MAX_NAME_LEN 50
#define MIN_CRITICALITY 1
#define MAX_CRITICALITY 5

// Component health status
typedef enum {
    STATUS_OPERATIONAL,
    STATUS_FAULTY
} ComponentStatus;

// Adjacency list node
typedef struct AdjListNode {
    int destIndex;
    struct AdjListNode *next;
} AdjListNode;

// Component information
typedef struct {
    char name[MAX_NAME_LEN];
    ComponentStatus status;
    int criticality;
    AdjListNode *head;
} ComponentVertex;

// Machine graph
typedef struct {
    ComponentVertex vertices[MAX_VERTICES];
    int numVertices;
    char machineName[MAX_NAME_LEN];
```

```
} MachineGraph;

// Circular queue for BFS
typedef struct {
    int items[MAX_VERTICES];
    int front;
    int rear;
    int count;
} Queue;



// INPUT FUNCTIONS

void clearInputBuffer(void)
{
    int ch;

    while ((ch = getchar()) != '\n' && ch != EOF);
}

void readString(char *str, int size)
{
    if (fgets(str, size, stdin) != NULL) {
        size_t len = strcspn(str, "\n");

        if (str[len] == '\n') {
            str[len] = '\0';
        }
else {
            clearInputBuffer();
        }
    }
else {
```

```
        str[0] = '\0';
    }
}
// QUEUE FUNCTIONS
void initQueue(Queue *q)
{
    q->front = 0;
    q->rear = -1;
    q->count = 0;
}

bool isQueueEmpty(const Queue *q)
{
    return q->count == 0;
}

bool isQueueFull(const Queue *q)
{
    return q->count == MAX_VERTICES;
}

bool enqueue(Queue *q, int value)
{
    if (isQueueFull(q))
        return false;

    q->rear = (q->rear + 1) % MAX_VERTICES;
    q->items[q->rear] = value;
    q->count++;

    return true;
}
```

```
int dequeue(Queue *q)
{
    int value;

    if (isQueueEmpty(q))
        return -1;

    value = q->items[q->front];
    q->front = (q->front + 1) % MAX_VERTICES;
    q->count--;

    return value;
}

//UTILITY FUNCTIONS

int caseInsensitiveCompare(const char *s1, const char *s2)
{
    while (*s1 && *s2) {
        int diff = tolower((unsigned char)*s1)
                - tolower((unsigned char)*s2);

        if (diff != 0)
            return diff;
        s1++;
        s2++;
    }
    return tolower((unsigned char)*s1)
         - tolower((unsigned char)*s2);
}
const char *getStatusString(ComponentStatus status)
{
    if (status == STATUS_FAULTY)
```

```
        return "FAULTY";

    return "OPERATIONAL";
}
int findComponentIndex(const MachineGraph *graph, const char *name)
{
    int i;

    for (i = 0; i < graph->numVertices; i++) {
        if (caseInsensitiveCompare(
                graph->vertices[i].name, name) == 0) {
            return i;
        }
    }

    return -1;
}



//GRAPH FUNCTIONS

void initGraph(MachineGraph *graph, const char *machineName)
{
    int i;

    graph->numVertices = 0;

    strncpy(graph->machineName, machineName, MAX_NAME_LEN - 1);
    graph->machineName[MAX_NAME_LEN - 1] = '\0';

    for (i = 0; i < MAX_VERTICES; i++) {
        graph->vertices[i].name[0] = '\0';
        graph->vertices[i].status = STATUS_OPERATIONAL;
```

```
        graph->vertices[i].criticality = 1;
        graph->vertices[i].head = NULL;
    }
}

void freeGraph(MachineGraph *graph)
{
    int i;

    for (i = 0; i < graph->numVertices; i++) {
        AdjListNode *current = graph->vertices[i].head;

        while (current != NULL) {
            AdjListNode *temp = current;
            current = current->next;
            free(temp);
        }

        graph->vertices[i].head = NULL;
    }

    graph->numVertices = 0;
}

int addComponent(MachineGraph *graph,
                 const char *name,
                 int criticality)
{
    int index;

    if (graph->numVertices >= MAX_VERTICES) {
        printf("\nError: Maximum of %d components allowed.\n",
               MAX_VERTICES);
```

```
    return -1;
}

if (strlen(name) == 0) {
    printf("\nError: Component name cannot be empty.\n");
    return -1;
}

if (findComponentIndex(graph, name) != -1) {
    printf("\nError: Component already exists.\n");
    return -1;
}

if (criticality < MIN_CRITICALITY ||
    criticality > MAX_CRITICALITY) {

    printf("Invalid criticality. Default value 3 used.\n");
    criticality = 3;
}

index = graph->numVertices;

strncpy(graph->vertices[index].name,
        name,
        MAX_NAME_LEN - 1);

graph->vertices[index].name[MAX_NAME_LEN - 1] = '\0';
graph->vertices[index].status = STATUS_OPERATIONAL;
graph->vertices[index].criticality = criticality;
graph->vertices[index].head = NULL;

graph->numVertices++;
```

```
    return index;
}

bool addDependency(MachineGraph *graph,
                   const char *sourceName,
                   const char *destinationName)
{
    int source;
    int destination;
    AdjListNode *current;
    AdjListNode *newNode;

    source = findComponentIndex(graph, sourceName);
    destination = findComponentIndex(graph, destinationName);

    if (source == -1 || destination == -1) {
        printf("\nError: One or both components were not found.\n");
        return false;
    }

    if (source == destination) {
        printf("\nError: A component cannot depend on itself.\n");
        return false;
    }

    current = graph->vertices[source].head;

    while (current != NULL) {
        if (current->destIndex == destination) {
            printf("\nError: Dependency already exists.\n");
            return false;
        }
```

```
        current = current->next;
    }

    newNode = (AdjListNode *)malloc(sizeof(AdjListNode));

    if (newNode == NULL) {
        printf("\nError: Memory allocation failed.\n");
        return false;
    }
    newNode->destIndex = destination;
    newNode->next = graph->vertices[source].head;
    graph->vertices[source].head = newNode;
    return true;
}
//SAMPLE MACHINE
void loadSampleMachine(MachineGraph *graph)
{
    freeGraph(graph);

    initGraph(graph, "Sample Industrial Air Compressor");

    addComponent(graph, "Bearing", 3);
    addComponent(graph, "Cooling Fan", 2);
    addComponent(graph, "Cooling System", 3);
    addComponent(graph, "Motor", 5);
    addComponent(graph, "Shaft", 4);
    addComponent(graph, "Control Unit", 5);
    addComponent(graph, "Compressor Unit", 5);
    addComponent(graph, "Pressure Sensor", 2);


// A -> B means B depends on A
```

```
    addDependency(graph, "Bearing", "Motor");
    addDependency(graph, "Cooling Fan", "Cooling System");
    addDependency(graph, "Cooling System", "Motor");
    addDependency(graph, "Cooling System", "Compressor Unit");
    addDependency(graph, "Motor", "Shaft");
    addDependency(graph, "Motor", "Control Unit");
    addDependency(graph, "Shaft", "Compressor Unit");
    addDependency(graph, "Control Unit", "Compressor Unit");
    addDependency(graph, "Compressor Unit", "Pressure Sensor");
}
// DISPLAY FUNCTIONS
void displayMachineStatus(const MachineGraph *graph)
{
    int i;
    printf("\n--------------------------------------------------------------\n");
    printf("MACHINE HEALTH REPORT\n");
    printf("Machine: %s\n", graph->machineName);
    printf("--------------------------------------------------------------\n");

    printf("%-4s %-25s %-12s %-15s\n",
           "ID", "Component", "Criticality", "Status");
    printf("--------------------------------------------------------------\n");

    for (i = 0; i < graph->numVertices; i++) {
        printf("%-4d %-25s %-12d %-15s\n",
               i + 1,
               graph->vertices[i].name,
               graph->vertices[i].criticality,
               getStatusString(graph->vertices[i].status));
    }

    printf("--------------------------------------------------------------\n");
}
```

```
void displayAdjacencyList(const MachineGraph *graph)
{
   int i;

   printf("\n------------------------------------------------------------\n");
   printf("MACHINE DEPENDENCY GRAPH\n");
   printf("------------------------------------------------------------\n");

   for (i = 0; i < graph->numVertices; i++) {

      AdjListNode *current =
         graph->vertices[i].head;

      printf("%s (Criticality %d) -> ",
            graph->vertices[i].name,
            graph->vertices[i].criticality);

      if (current == NULL) {
         printf("No downstream components");
      }

      while (current != NULL) {
         printf("%s",
               graph->vertices[current->destIndex].name);

         if (current->next != NULL)
            printf(" -> ");

         current = current->next;
      }

      printf("\n");
```

```
    }
    printf("------------------------------------------------------------\n");
}
//BFS FAILURE IMPACT ANALYSIS
void analyzeFailureBFS(const MachineGraph *graph, int startIndex)
{
    bool visited[MAX_VERTICES] = { false };
    int level[MAX_VERTICES];
    Queue queue;
    int i;
    int currentIndex;
    int totalAffected = 0;
    int criticalAffected = 0;
    int maxLevel = 0;
    int impactScore = 0;

    for (i = 0; i < MAX_VERTICES; i++)
        level[i] = -1;

    initQueue(&queue);

    visited[startIndex] = true;
    level[startIndex] = 0;

    enqueue(&queue, startIndex);

    while (!isQueueEmpty(&queue)) {

        currentIndex = dequeue(&queue);

        if (currentIndex != startIndex) {
            totalAffected++;
```

```
        impactScore +=
            graph->vertices[currentIndex].criticality;

        if (graph->vertices[currentIndex].criticality >= 4)
            criticalAffected++;
    }

    AdjListNode *current =
        graph->vertices[currentIndex].head;

    while (current != NULL) {

        int nextIndex = current->destIndex;

        if (!visited[nextIndex]) {

            visited[nextIndex] = true;

            level[nextIndex] =
                level[currentIndex] + 1;

            if (level[nextIndex] > maxLevel)
                maxLevel = level[nextIndex];

            enqueue(&queue, nextIndex);
        }
        current = current->next;
    }
}
printf("BFS FAILURE IMPACT ANALYSIS\n");
printf("------------------------------------------------------------\n");
printf("Failed Component: %s\n",
       graph->vertices[startIndex].name);
```

```
    printf("Criticality: %d\n",
           graph->vertices[startIndex].criticality);
    printf("\nAffected Components:\n");

    for (i = 1; i <= maxLevel; i++) {

        printf("\nLevel %d:\n", i);

        for (int j = 0; j < graph->numVertices; j++) {

            if (level[j] == i) {

                printf("  %s - AT RISK - Criticality %d\n",
                       graph->vertices[j].name,
                       graph->vertices[j].criticality);
            }
        }
    }
    if (totalAffected == 0) {
        printf("\nNo downstream components are affected.\n");
    }
    printf("\n------------------------------------------------------------\n");
    printf("Total Components At Risk : %d\n", totalAffected);
    printf("High Criticality At Risk : %d\n", criticalAffected);
    printf("Maximum Impact Level     : %d\n", maxLevel);
    printf("Cumulative Impact Score  : %d\n", impactScore);
    printf("------------------------------------------------------------\n");
}

//   DFS FAILURE PATH ANALYSIS
void dfsTrace(const MachineGraph *graph,
              int current,
              bool visited[],
```

```
            int path[],
            int pathLength,
            int *pathCount)
{
    AdjListNode *edge;
    bool hasNext = false;
    visited[current] = true;
    path[pathLength] = current;
    pathLength++;
    edge = graph->vertices[current].head;

    while (edge != NULL) {
        int next = edge->destIndex;

        if (!visited[next]) {
            hasNext = true;
            dfsTrace(graph, next, visited, path, pathLength,  pathCount);
        }
        edge = edge->next;
    }
    // Print path if it reaches a terminal node or has no unvisited downstream nodes
    if (!hasNext && pathLength > 1) {
        int i;
        (*pathCount)++;
        printf("Path %d: ", *pathCount);

        for (i = 0; i < pathLength; i++) {
            printf("%s", graph->vertices[path[i]].name);
            if (i < pathLength - 1) {
                printf(" -> ");
            }
        }
        printf("\n");
```

```
    }
    visited[current] = false;
}

void traceFailureDFS(const MachineGraph *graph, int startIndex)
{
    bool visited[MAX_VERTICES] = { false };
    int path[MAX_VERTICES];
    int pathCount = 0;

    printf("\n-------------------------------------------------------------\n");
    printf("DFS FAILURE PROPAGATION PATHS\n");
    printf("-------------------------------------------------------------\n");
    printf("Starting Component: %s\n\n",
           graph->vertices[startIndex].name);

    dfsTrace(graph, startIndex, visited, path, 0, &pathCount);
    if (pathCount == 0) {
        printf("No downstream propagation path found.\n");
    }
    printf("-------------------------------------------------------------\n");
}

//UPSTREAM DEPENDENCY ANALYSIS
void findUpstreamDependencies(const MachineGraph *graph,
                              const char *targetName)
{
    int targetIndex;
    int i;
    int count = 0;

    targetIndex =
        findComponentIndex(graph, targetName);
```

```
    if (targetIndex == -1) {
        printf("\nComponent not found.\n");
        return;
    }
    printf("\n------------------------------------------------------------\n");
    printf("UPSTREAM DEPENDENCY ANALYSIS\n");
    printf("------------------------------------------------------------\n");
    printf("Target Component: %s\n\n",
           graph->vertices[targetIndex].name);

    printf("Components directly feeding this component:\n");
    for (i = 0; i < graph->numVertices; i++) {
        AdjListNode *current =
            graph->vertices[i].head;
        while (current != NULL) {
            if (current->destIndex == targetIndex) {
                printf("  %s - Criticality %d\n",
                       graph->vertices[i].name,
                       graph->vertices[i].criticality);
                count++;
                break;
            }
            current = current->next;
        }
    }
    if (count == 0) {
        printf("  No upstream components found.\n");
    }
    printf("------------------------------------------------------------\n");
}

// COMPONENT SEARCH
```

```
void searchComponent(const MachineGraph *graph)
{
    char name[MAX_NAME_LEN];
    int index;
    printf("\nEnter component name: ");
    readString(name, MAX_NAME_LEN);
    index = findComponentIndex(graph, name);
    if (index == -1) {
        printf("\nComponent not found.\n");
        return;
    }
    printf("\n------------------------------------------------------------\n");
    printf("COMPONENT INFORMATION\n");
    printf("------------------------------------------------------------\n");
    printf("Name       : %s\n",
           graph->vertices[index].name);
    printf("Criticality : %d\n",
           graph->vertices[index].criticality);
    printf("Status     : %s\n",
           getStatusString(graph->vertices[index].status));
    printf("------------------------------------------------------------\n");
}

//RESET MACHINE
void resetMachine(MachineGraph *graph)
{
    int i;
    for (i = 0; i < graph->numVertices; i++) {
        graph->vertices[i].status =
            STATUS_OPERATIONAL;
    }
    printf("\nAll components have been reset to OPERATIONAL.\n");
}
```

```
// ADD COMPONENT
void addNewComponent(MachineGraph *graph)
{
   char name[MAX_NAME_LEN];
   int criticality;
   printf("\nEnter component name: ");
   readString(name, MAX_NAME_LEN);
   printf("Enter criticality (1-5): ");

   if (scanf("%d", &criticality) != 1) {
      clearInputBuffer();
      printf("Invalid input. Criticality set to 3.\n");
      criticality = 3;
   }
else {
      clearInputBuffer();
   }
   if (addComponent(graph, name, criticality) != -1) {
      printf("\nComponent '%s' added successfully.\n",
            name);
   }
}
//   ADD DEPENDENCY
void addNewDependency(MachineGraph *graph)
{
   char source[MAX_NAME_LEN];
   char destination[MAX_NAME_LEN];

   printf("\nEnter source component: ");
   readString(source, MAX_NAME_LEN);
   printf("Enter dependent component: ");
   readString(destination, MAX_NAME_LEN);
```

```
    if (addDependency(graph, source, destination)) {
        printf("\nDependency added:\n");
        printf("%s -> %s\n",
               source,
               destination);
    }
}

//  CREATE CUSTOM MACHINE
void createCustomMachine(MachineGraph *graph)
{
    char machineName[MAX_NAME_LEN];
    int numberOfComponents;
    int i;
    freeGraph(graph);
    printf("\nEnter machine name: ");
    readString(machineName, MAX_NAME_LEN);

    if (strlen(machineName) == 0) {
        strcpy(machineName, "Custom Industrial Machine");
    }

    initGraph(graph, machineName);
    printf("\nHow many components do you want to add? ");
    if (scanf("%d", &numberOfComponents) != 1) {
        clearInputBuffer();
        numberOfComponents = 0;
    }
else {
        clearInputBuffer();
    }
```

```
    if (numberOfComponents < 0)
        numberOfComponents = 0;

    if (numberOfComponents > MAX_VERTICES)
        numberOfComponents = MAX_VERTICES;

    for (i = 0; i < numberOfComponents; i++) {
        printf("\nComponent %d\n", i + 1);
        addNewComponent(graph);
    }
    printf("\nCustom machine created successfully.\n");
}

//   MACHINE CONFIGURATION

void selectMachineConfiguration(MachineGraph *graph)
{
    int choice;
    printf("\n-----------------------------------------------------------\n");
    printf("MACHINE CONFIGURATION\n");
    printf("-----------------------------------------------------------\n");
    printf("1. Load Sample Air Compressor\n");
    printf("2. Create Custom Machine\n");
    printf("-----------------------------------------------------------\n");
    printf("Enter choice: ");
    if (scanf("%d", &choice) != 1) {
        clearInputBuffer();
        choice = 1;
    } else {
        clearInputBuffer();
    }
    if (choice == 2) {
        createCustomMachine(graph);
```

```
    }
    else {
        loadSampleMachine(graph);
        printf("\nSample air compressor loaded successfully.\n");
    }
}


// REGISTER FAULT
void registerFault(MachineGraph *graph)
{
    char name[MAX_NAME_LEN];
    int index;

    printf("\nEnter faulty component name: ");
    readString(name, MAX_NAME_LEN);
    index = findComponentIndex(graph, name);

    if (index == -1) {
        printf("\nComponent not found.\n");
        return;
    }
    graph->vertices[index].status = STATUS_FAULTY;
    printf("\nFault registered for '%s'.\n", graph->vertices[index].name);
    printf("\nRunning BFS impact analysis...\n");
    analyzeFailureBFS(graph, index);
    printf("\nRunning DFS propagation analysis...\n");
    traceFailureDFS(graph, index);
}

int main(void)
{
    MachineGraph machine;
```

```
int choice;
initGraph(&machine, "Unconfigured Machine");
printf("\n----------------------------------------------------------\n");
printf("MACHINE SAATHI \n");
printf("Industrial Machine Dependency and Fault Analyzer\n");
printf("----------------------------------------------------------\n");
selectMachineConfiguration(&machine);

while (1) {
printf("\n----------------------------------------------------------\n");
printf("MACHINE SAATHI\n");
printf("Machine: %s\n", machine.machineName);
printf("----------------------------------------------------------\n");
printf("1. Display Machine Health\n");
printf("2. View Dependency Graph\n");
printf("3. Register Component Fault\n");
printf("4. BFS Fault Impact Analysis\n");
printf("5. DFS Fault Propagation Paths\n");
printf("6. Upstream Dependency Analysis\n");
printf("7. Search Component\n");
printf("8. Add New Component\n");
printf("9. Add Dependency\n");
printf("10. Reset Machine Health\n");
printf("11. Change Machine Configuration\n");
printf("12. Exit\n");
printf("----------------------------------------------------------\n");
printf("Enter choice: ");
   if (scanf("%d", &choice) != 1) {
      clearInputBuffer();
      printf("\nInvalid input. Please enter a number.\n");
      continue;
   }
   clearInputBuffer();
```

```
switch (choice) {
    case 1:
        displayMachineStatus(&machine);
        break;

    case 2:
        displayAdjacencyList(&machine);
        break;
    case 3:
        registerFault(&machine);
        break;

    case 4:
    {
        char name[MAX_NAME_LEN];
        int index;
        printf("\nEnter component name: ");
        readString(name, MAX_NAME_LEN);
        index =
            findComponentIndex(&machine, name);
        if (index == -1) {
            printf("\nComponent not found.\n");
        }
        else {
            analyzeFailureBFS(&machine, index);
        }
        break;
    }

    case 5:
    {
        char name[MAX_NAME_LEN];
```

```
    int index;
    printf("\nEnter component name: ");
    readString(name, MAX_NAME_LEN);
    index =
        findComponentIndex(&machine, name);
    if (index == -1) {
        printf("\nComponent not found.\n");
    }
    else {
        traceFailureDFS(&machine, index);
    }
    break;
}

case 6:
{
    char name[MAX_NAME_LEN];

    printf("\nEnter component name: ");
    readString(name, MAX_NAME_LEN);
    findUpstreamDependencies( &machine, name);
    break;
}
case 7:
    searchComponent(&machine);
    break;
case 8:
    addNewComponent(&machine);
    break;
case 9:
    addNewDependency(&machine);
    break;
case 10:
```

```
            resetMachine(&machine);
            break;
        case 11:
            selectMachineConfiguration(&machine);
            break;
        case 12:
            freeGraph(&machine);
            printf("\nMemory released successfully.\n");
            printf("Machine Saathi closed.\n");
            return 0;
        default:
            printf("\nInvalid choice. Please select 1-12.\n");
      }
   }
   return 0;
}
```

8. Result Analysis with Output Screenshots

```
------------------------------------------------------------
MACHINE SAATHI
Industrial Machine Dependency and Fault Analyzer
------------------------------------------------------------

------------------------------------------------------------
MACHINE CONFIGURATION
------------------------------------------------------------
1. Load Sample Air Compressor
2. Create Custom Machine
------------------------------------------------------------
Enter choice: 1

Sample air compressor loaded successfully.

------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 1

------------------------------------------------------------
MACHINE HEALTH REPORT
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
ID   Component                Criticality  Status
------------------------------------------------------------
1    Bearing                  3            OPERATIONAL
2    Cooling Fan              2            OPERATIONAL
3    Cooling System           3            OPERATIONAL
4    Motor                    5            OPERATIONAL
5    Shaft                    4            OPERATIONAL
6    Control Unit             5            OPERATIONAL
7    Compressor Unit          5            OPERATIONAL
8    Pressure Sensor          2            OPERATIONAL
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 2

------------------------------------------------------------
MACHINE DEPENDENCY GRAPH
------------------------------------------------------------
Bearing (Criticality 3) -> Motor
Cooling Fan (Criticality 2) -> Cooling System
Cooling System (Criticality 3) -> Compressor Unit -> Motor
Motor (Criticality 5) -> Control Unit -> Shaft
Shaft (Criticality 4) -> Compressor Unit
Control Unit (Criticality 5) -> Compressor Unit
Compressor Unit (Criticality 5) -> Pressure Sensor
Pressure Sensor (Criticality 2) -> No downstream components
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 3

Enter faulty component name: Shaft

Fault registered for 'Shaft'.

Running BFS impact analysis...
BFS FAILURE IMPACT ANALYSIS
------------------------------------------------------------
Failed Component: Shaft
Criticality: 4

Affected Components:

Level 1:
  Compressor Unit - AT RISK - Criticality 5

Level 2:
  Pressure Sensor - AT RISK - Criticality 2

------------------------------------------------------------
Total Components At Risk : 2
High Criticality At Risk : 1
Maximum Impact Level     : 2
Cumulative Impact Score  : 7
------------------------------------------------------------

Running DFS propagation analysis...

------------------------------------------------------------
DFS FAILURE PROPAGATION PATHS
------------------------------------------------------------
Starting Component: Shaft

Path 1: Shaft -> Compressor Unit -> Pressure Sensor
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 1

------------------------------------------------------------
MACHINE HEALTH REPORT
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
ID   Component                Criticality  Status
------------------------------------------------------------
1    Bearing                  3            OPERATIONAL
2    Cooling Fan              2            OPERATIONAL
3    Cooling System           3            OPERATIONAL
4    Motor                    5            OPERATIONAL
5    Shaft                    4            FAULTY
6    Control Unit             5            OPERATIONAL
7    Compressor Unit          5            OPERATIONAL
8    Pressure Sensor          2            OPERATIONAL
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 4

Enter component name: Bearing
BFS FAILURE IMPACT ANALYSIS
------------------------------------------------------------
Failed Component: Bearing
Criticality: 3

Affected Components:

Level 1:
  Motor - AT RISK - Criticality 5

Level 2:
  Shaft - AT RISK - Criticality 4
  Control Unit - AT RISK - Criticality 5

Level 3:
  Compressor Unit - AT RISK - Criticality 5

Level 4:
  Pressure Sensor - AT RISK - Criticality 2

------------------------------------------------------------
Total Components At Risk : 5
High Criticality At Risk : 4
Maximum Impact Level     : 4
Cumulative Impact Score  : 21
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 5

Enter component name: Cooling Fan

------------------------------------------------------------
DFS FAILURE PROPAGATION PATHS
------------------------------------------------------------
Starting Component: Cooling Fan

Path 1: Cooling Fan -> Cooling System -> Compressor Unit -> Pressure Sensor
Path 2: Cooling Fan -> Cooling System -> Motor -> Control Unit -> Compressor Unit -> Pressure Sensor
Path 3: Cooling Fan -> Cooling System -> Motor -> Shaft -> Compressor Unit -> Pressure Sensor
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 6

Enter component name: Motor

------------------------------------------------------------
UPSTREAM DEPENDENCY ANALYSIS
------------------------------------------------------------
Target Component: Motor

Components directly feeding this component:
  Bearing - Criticality 3
  Cooling System - Criticality 3
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 7

Enter component name: Cooling System

------------------------------------------------------------
COMPONENT INFORMATION
------------------------------------------------------------
Name        : Cooling System
Criticality : 3
Status      : OPERATIONAL
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 8

Enter component name: Hydraulic Pump
Enter criticality (1-5): 4

Component 'Hydraulic Pump' added successfully.
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Hydraulic Pump
Enter dependent component: Control Unit

Dependency added:
Hydraulic Pump -> Control Unit
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 1

------------------------------------------------------------
MACHINE HEALTH REPORT
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
ID   Component                 Criticality  Status
------------------------------------------------------------
1    Bearing                   3            OPERATIONAL
2    Cooling Fan               2            OPERATIONAL
3    Cooling System            3            OPERATIONAL
4    Motor                     5            OPERATIONAL
5    Shaft                     4            FAULTY
6    Control Unit              5            OPERATIONAL
7    Compressor Unit           5            OPERATIONAL
8    Pressure Sensor           2            OPERATIONAL
9    Hydraulic Pump            4            OPERATIONAL
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 10

All components have been reset to OPERATIONAL.

------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 1

------------------------------------------------------------
MACHINE HEALTH REPORT
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
ID   Component                Criticality  Status
------------------------------------------------------------
1    Bearing                  3            OPERATIONAL
2    Cooling Fan              2            OPERATIONAL
3    Cooling System           3            OPERATIONAL
4    Motor                    5            OPERATIONAL
5    Shaft                    4            OPERATIONAL
6    Control Unit             5            OPERATIONAL
7    Compressor Unit          5            OPERATIONAL
8    Pressure Sensor          2            OPERATIONAL
9    Hydraulic Pump           4            OPERATIONAL
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Sample Industrial Air Compressor
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 11

------------------------------------------------------------
MACHINE CONFIGURATION
------------------------------------------------------------
1. Load Sample Air Compressor
2. Create Custom Machine
------------------------------------------------------------
Enter choice: 2

Enter machine name: Industrial Water Pumping System

How many components do you want to add? 8

Component 1

Enter component name: Water Pump
Enter criticality (1-5): 5

Component 'Water Pump' added successfully.

Component 2

Enter component name: Electric Motor
Enter criticality (1-5): 5

Component 'Electric Motor' added successfully.

Component 3

Enter component name: Impeller
Enter criticality (1-5): 4

Component 'Impeller' added successfully.
```

```
Component 4

Enter component name: Shaft
Enter criticality (1-5): 4

Component 'Shaft' added successfully.

Component 5

Enter component name: Bearing
Enter criticality (1-5): 3

Component 'Bearing' added successfully.

Component 6

Enter component name: Control Unit
Enter criticality (1-5): 5

Component 'Control Unit' added successfully.

Component 7

Enter component name: Pressure Sensor
Enter criticality (1-5): 3

Component 'Pressure Sensor' added successfully.

Component 8

Enter component name: Water Flow Sensor
Enter criticality (1-5): 2

Component 'Water Flow Sensor' added successfully.

Custom machine created successfully.
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 1

------------------------------------------------------------
MACHINE HEALTH REPORT
Machine: Industrial Water Pumping System
------------------------------------------------------------
ID   Component                 Criticality  Status
------------------------------------------------------------
1    Water Pump                5            OPERATIONAL
2    Electric Motor            5            OPERATIONAL
3    Impeller                  4            OPERATIONAL
4    Shaft                     4            OPERATIONAL
5    Bearing                   3            OPERATIONAL
6    Control Unit              5            OPERATIONAL
7    Pressure Sensor           3            OPERATIONAL
8    Water Flow Sensor         2            OPERATIONAL
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Bearing
Enter dependent component: Shaft

Dependency added:
Bearing -> Shaft

------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Shaft
Enter dependent component: Impeller

Dependency added:
Shaft -> Impeller

------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Impeller
Enter dependent component: Water Pump

Dependency added:
Impeller -> Water Pump
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Electric Motor
Enter dependent component: Shaft

Dependency added:
Electric Motor -> Shaft

------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Control Unit
Enter dependent component: Electric Motor

Dependency added:
Control Unit -> Electric Motor
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Water Pump
Enter dependent component: Pressure Sensor

Dependency added:
Water Pump -> Pressure Sensor

------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Water Pump
Enter dependent component: Water Flow Sensor

Dependency added:
Water Pump -> Water Flow Sensor
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 2

------------------------------------------------------------
MACHINE DEPENDENCY GRAPH
------------------------------------------------------------
Water Pump (Criticality 5) -> Water Flow Sensor -> Pressure Sensor
Electric Motor (Criticality 5) -> Shaft
Impeller (Criticality 4) -> Water Pump
Shaft (Criticality 4) -> Impeller
Bearing (Criticality 3) -> Shaft
Control Unit (Criticality 5) -> Electric Motor
Pressure Sensor (Criticality 3) -> No downstream components
Water Flow Sensor (Criticality 2) -> No downstream components
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 3

Enter faulty component name: Water Flow Sensor

Fault registered for 'Water Flow Sensor'.

Running BFS impact analysis...
BFS FAILURE IMPACT ANALYSIS
------------------------------------------------------------
Failed Component: Water Flow Sensor
Criticality: 2

Affected Components:

No downstream components are affected.

------------------------------------------------------------
Total Components At Risk : 0
High Criticality At Risk : 0
Maximum Impact Level     : 0
Cumulative Impact Score  : 0
------------------------------------------------------------

Running DFS propagation analysis...

------------------------------------------------------------
DFS FAILURE PROPAGATION PATHS
------------------------------------------------------------
Starting Component: Water Flow Sensor

No downstream propagation path found.
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 1

------------------------------------------------------------
MACHINE HEALTH REPORT
Machine: Industrial Water Pumping System
------------------------------------------------------------
ID   Component                Criticality  Status
------------------------------------------------------------
1    Water Pump               5            OPERATIONAL
2    Electric Motor           5            OPERATIONAL
3    Impeller                 4            OPERATIONAL
4    Shaft                    4            OPERATIONAL
5    Bearing                  3            OPERATIONAL
6    Control Unit             5            OPERATIONAL
7    Pressure Sensor          3            OPERATIONAL
8    Water Flow Sensor        2            FAULTY
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 4

Enter component name: Shaft
BFS FAILURE IMPACT ANALYSIS
------------------------------------------------------------
Failed Component: Shaft
Criticality: 4

Affected Components:

Level 1:
  Impeller - AT RISK - Criticality 4

Level 2:
  Water Pump - AT RISK - Criticality 5

Level 3:
  Pressure Sensor - AT RISK - Criticality 3
  Water Flow Sensor - AT RISK - Criticality 2

------------------------------------------------------------
Total Components At Risk : 4
High Criticality At Risk : 2
Maximum Impact Level     : 3
Cumulative Impact Score  : 14
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 5

Enter component name: Water Pump

------------------------------------------------------------
DFS FAILURE PROPAGATION PATHS
------------------------------------------------------------
Starting Component: Water Pump

Path 1: Water Pump -> Water Flow Sensor
Path 2: Water Pump -> Pressure Sensor
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 6

Enter component name: Electric Motor

------------------------------------------------------------
UPSTREAM DEPENDENCY ANALYSIS
------------------------------------------------------------
Target Component: Electric Motor

Components directly feeding this component:
  Control Unit - Criticality 5
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 7

Enter component name: Control Unit

------------------------------------------------------------
COMPONENT INFORMATION
------------------------------------------------------------
Name        : Control Unit
Criticality : 5
Status      : OPERATIONAL
------------------------------------------------------------

------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 8

Enter component name: Water Filter
Enter criticality (1-5): 3

Component 'Water Filter' added successfully.
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 9

Enter source component: Water Filter
Enter dependent component: Water Pump

Dependency added:
Water Filter -> Water Pump

------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 1

------------------------------------------------------------
MACHINE HEALTH REPORT
Machine: Industrial Water Pumping System
------------------------------------------------------------
ID   Component                Criticality  Status
------------------------------------------------------------
1    Water Pump               5            OPERATIONAL
2    Electric Motor           5            OPERATIONAL
3    Impeller                 4            OPERATIONAL
4    Shaft                    4            OPERATIONAL
5    Bearing                  3            OPERATIONAL
6    Control Unit             5            OPERATIONAL
7    Pressure Sensor          3            OPERATIONAL
8    Water Flow Sensor        2            FAULTY
9    Water Filter             3            OPERATIONAL
------------------------------------------------------------
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 2

------------------------------------------------------------
MACHINE DEPENDENCY GRAPH
------------------------------------------------------------
Water Pump (Criticality 5) -> Water Flow Sensor -> Pressure Sensor
Electric Motor (Criticality 5) -> Shaft
Impeller (Criticality 4) -> Water Pump
Shaft (Criticality 4) -> Impeller
Bearing (Criticality 3) -> Shaft
Control Unit (Criticality 5) -> Electric Motor
Pressure Sensor (Criticality 3) -> No downstream components
Water Flow Sensor (Criticality 2) -> No downstream components
Water Filter (Criticality 3) -> Water Pump
------------------------------------------------------------

------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 10

All components have been reset to OPERATIONAL.
```

```
------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 1

------------------------------------------------------------
MACHINE HEALTH REPORT
Machine: Industrial Water Pumping System
------------------------------------------------------------
ID   Component                Criticality  Status
------------------------------------------------------------
1    Water Pump               5            OPERATIONAL
2    Electric Motor           5            OPERATIONAL
3    Impeller                 4            OPERATIONAL
4    Shaft                    4            OPERATIONAL
5    Bearing                  3            OPERATIONAL
6    Control Unit             5            OPERATIONAL
7    Pressure Sensor          3            OPERATIONAL
8    Water Flow Sensor        2            OPERATIONAL
9    Water Filter             3            OPERATIONAL
------------------------------------------------------------

------------------------------------------------------------
MACHINE SAATHI
Machine: Industrial Water Pumping System
------------------------------------------------------------
1. Display Machine Health
2. View Dependency Graph
3. Register Component Fault
4. BFS Fault Impact Analysis
5. DFS Fault Propagation Paths
6. Upstream Dependency Analysis
7. Search Component
8. Add New Component
9. Add Dependency
10. Reset Machine Health
11. Change Machine Configuration
12. Exit
------------------------------------------------------------
Enter choice: 12

Memory released successfully.
Machine Saathi closed.

--------------------------------
Process exited after 614.2 seconds with return value 0
Press any key to continue . . .
```

9. Result Analysis

The implemented program demonstrates how a directed graph can represent component dependencies and support machine failure-impact analysis. BFS identifies downstream components reachable from a reported faulty component and groups them according to their shortest graph-distance levels, distinguishing direct dependencies from indirect ones.

DFS complements BFS by tracing possible simple paths through the dependency graph. This provides additional information about the different dependency routes connecting the faulty component to

downstream components. The results are based on the graph configured by the user and represent potential impact according to the defined dependencies, rather than confirmed physical failure propagation.

The program supports user-defined machine configurations, component searches, dependency lookup, status management, and graph display. Its fixed capacity is 50 components, making it suitable for demonstrating graph-based dependency analysis on small machine models. BFS has a time complexity of (O(V+E)), while the DFS path-enumeration operation can take exponential time in the worst case.

Overall, the project demonstrates the practical application of adjacency lists, queues, recursion, BFS, and DFS to a structured machine-dependency problem. It provides an interpretable foundation for failure-impact analysis without requiring sensors, machine-learning models, or real-time monitoring.

10. Conclusion

Machine Saathi demonstrates how graph data structures and traversal algorithms can be applied to model machine-component dependencies and analyse the potential impact of a component failure. The adjacency-list representation supports user-defined machine configurations, while BFS identifies reachable downstream components by graph distance and DFS explores possible dependency paths.

The project provides a simple, interpretable, and menu-driven approach to understanding interconnected machine components. However, its results depend on the accuracy of the dependencies entered by the user. The program does not use sensor readings, predict actual failures, or establish the physical timing of failure propagation. Future extensions could incorporate real machine data, more detailed dependency relationships, and sensor-based monitoring to support more advanced maintenance analysis.

11. References

[1] Y. Qin, H. Wang, and Y. Li, "Toward cognitive predictive maintenance: A survey of graph-based approaches," Computers in Industry, vol. 144, 2023.

[2] F. Yang, "Signed directed graph-based hierarchical modelling and fault propagation analysis for large-scale systems," IET Control Theory & Applications, vol. 7, no. 2, 2013, doi: 10.1049/iet-cta.2010.0660.

[3] X. Xue et al., "Hierarchical causal graph-based fault root cause diagnosis and propagation path identification for complex industrial process monitoring," IEEE Trans. Instrum. Meas., vol. 72, 2023, Art. no. 3513911.

[4] L. Xia, Y. Liang, J. Leng, and P. Zheng, "Maintenance planning recommendation of complex industrial equipment based on knowledge graph and graph neural network," Reliability Engineering & System Safety, vol. 232, 2023, Art. no. 109068.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022 (BFS, DFS, and adjacency-list representation).